

Arch Linux Productivity Machine — KDE Plasma

Dev • Writing • Browsing

Overview

This guide takes you from a blank drive to a fully configured productivity workstation. It assumes you've installed Arch before so it skips hand-holding on basics, but covers every decision point clearly. The stack is: **systemd-boot**, **btrfs with subvolumes**, **pipewire**, **KDE Plasma**, and a curated productivity toolset.

Estimated time: 2-3 hours from ISO to working desktop.

Phase 1 — Base Installation

1.1 Boot and network

```
bash

# Verify UEFI boot
ls /sys/firmware/efi/efivars

# Connect to wifi if needed
iwctl
    station wlan0 connect "YourSSID"
exit

# Sync time
timedatectl set-ntp true
```

1.2 Partition the disk

This layout uses btrfs for the root (giving you snapshots and easy rollback) and a separate EFI partition. Adjust `/dev/nvme0n1` to your actual disk.

```
bash

lsblk # confirm your disk name

gdisk /dev/nvme0n1
# Create two partitions:
#   p1: +512M, type EF00 (EFI)
#   p2: remainder, type 8300 (Linux)
```

Recommended partition layout:

Partition	Size	Type	Filesystem
/dev/nvme0n1p1	512 MB	EFI System	FAT32
/dev/nvme0n1p2	Remainder	Linux	btrfs

```
bash
```

```
mkfs.fat -F32 /dev/nvme0n1p1
mkfs.btrfs -L arch /dev/nvme0n1p2
```

1.3 Btrfs subvolumes

Using subvolumes lets you snapshot the system without capturing large volatile directories like `/var/log` or your home directory separately.

```
bash
```

```
mount /dev/nvme0n1p2 /mnt

btrfs subvolume create /mnt/@
btrfs subvolume create /mnt/@home
btrfs subvolume create /mnt/@var
btrfs subvolume create /mnt/@snapshots
btrfs subvolume create /mnt/@cache

umount /mnt
```

Mount with compression enabled (noticeable performance and space benefit on SSDs):

```
bash
```

```
mount -o noatime,compress=zstd,subvol=@ /dev/nvme0n1p2 /mnt

mkdir -p /mnt/{home,var,boot/efi,.snapshots}

mount -o noatime,compress=zstd,subvol=@home /dev/nvme0n1p2 /mnt/home
mount -o noatime,compress=zstd,subvol=@var /dev/nvme0n1p2 /mnt/var
mount -o noatime,compress=zstd,subvol=@snapshots /dev/nvme0n1p2 /mnt/.snapshots
mount /dev/nvme0n1p1 /mnt/boot/efi
```

1.4 Install base system

```
bash
```

```
pacstrap /mnt \
  base base-devel linux linux-headers linux-firmware \
  btrfs-progs \
  networkmanager \
  vim nano \
  git curl wget \
  man-db man-pages \
  sudo
```

For AMD CPU replace `intel-ucode` below with `amd-ucode`. Install the right one:

```
bash

# Intel:
pacstrap /mnt intel-ucode

# AMD:
pacstrap /mnt amd-ucode
```

1.5 Generate fstab

```
bash

genfstab -U /mnt >> /mnt/etc/fstab

# Review it – confirm all 5 mount points are present
cat /mnt/etc/fstab
```

1.6 Chroot and configure

```
bash

arch-chroot /mnt
```

```
bash
```

```
# Timezone
ln -sf /usr/share/zoneinfo/Asia/Kolkata /etc/localtime # adjust to yours
hwclock --systohc

# Locale
echo "en_US.UTF-8 UTF-8" >> /etc/locale.gen
locale-gen
echo "LANG=en_US.UTF-8" > /etc/locale.conf

# Hostname
echo "yourbox" > /etc/hostname
cat > /etc/hosts << EOF
127.0.0.1    localhost
::1         localhost
127.0.1.1    yourbox.localdomain yourbox
EOF

# Root password
passwd

# Create your user
useradd -m -G wheel,audio,video,storage,optical,network -s /bin/bash yourname
passwd yourname

# Enable sudo for wheel group
EDITOR=vim visudo
# Uncomment: %wheel ALL=(ALL:ALL) ALL
```

1.7 Bootloader — systemd-boot

Lighter than GRUB, faster, and handles microcode automatically.

```
bash
```

```
bootctl --path=/boot/efi install

# Get the UUID of your btrfs partition
blkid /dev/nvme0n1p2    # copy the UUID value

# Create boot entry
mkdir -p /boot/efi/loader/entries

cat > /boot/efi/loader/loader.conf << EOF
default arch.conf
timeout 3
console-mode max
editor no
EOF

cat > /boot/efi/loader/entries/arch.conf << EOF
title    Arch Linux
linux    /vmlinuz-linux
initrd   /intel-ucode.img
initrd   /initramfs-linux.img
options  root=UUID=YOUR-UUID-HERE rootflags=subvol=@ rw quiet splash
EOF

# Replace intel-ucode with amd-ucode if AMD
# Replace YOUR-UUID-HERE with the UUID from blkid
```

1.8 Initramfs and reboot

```
bash

mkinitcpio -P
exit
umount -R /mnt
reboot
```

Phase 2 — Post-Boot System Setup

Log in as your user after reboot.

2.1 Enable essential services

```
bash
```

```
sudo systemctl enable --now NetworkManager
sudo systemctl enable --now fstrim.timer          # weekly SSD TRIM
sudo systemctl enable --now systemd-timesyncd    # NTP
```

2.2 Pacman configuration

```
bash

sudo vim /etc/pacman.conf
```

Uncomment or add these:

```
ini

[options]
Color
VerbosePkgLists
ParallelDownloads = 5

# Uncomment multilib section at the bottom:
[multilib]
Include = /etc/pacman.d/mirrorlist
```

Refresh mirrors (pick the fastest ones for your region):

```
bash

sudo pacman -S reflector
sudo reflector --country India --latest 10 --sort rate --save /etc/pacman.d/mirrorli
# Change --country to match yours
sudo pacman -Syu
```

2.3 Install yay (AUR helper)

```
bash

cd /tmp
git clone https://aur.archlinux.org/yay.git
cd yay
makepkg -si
```

Phase 3 — KDE Plasma Desktop

3.1 GPU drivers

Install before Plasma to avoid headaches:

```
bash

# Intel integrated graphics:
sudo pacman -S mesa intel-media-driver vulkan-intel

# AMD:
sudo pacman -S mesa vulkan-radeon libva-mesa-driver

# NVIDIA (proprietary):
sudo pacman -S nvidia nvidia-utils nvidia-settings
# Add 'nvidia' to MODULES in /etc/mkinitcpio.conf then: sudo mkinitcpio -P
```

3.2 Plasma + display manager

```
bash

sudo pacman -S \
    plasma-meta \
    kde-applications-meta \
    sddm \
    xorg-server \
    xorg-xinit \
    pipewire \
    pipewire-alsa \
    pipewire-pulse \
    pipewire-jack \
    wireplumber \
    packagekit-qt6

sudo systemctl enable sddm
sudo systemctl enable --now bluetooth    # if you have BT hardware
```

If you don't want the full `kde-applications-meta` (it's large), use this minimal set instead:

```
bash
```

```
sudo pacman -S \  
  dolphin \      # file manager  
  konsole \      # terminal  
  kate \         # text editor  
  spectacle \    # screenshot  
  gwenview \     # image viewer  
  okular \       # document viewer  
  ark \          # archive manager  
  kcalc \        # calculator  
  krunner        # launcher
```

Reboot into Plasma:

```
bash  
  
reboot
```

3.3 KDE initial configuration

After logging into Plasma, do these first:

System Settings → Appearance

- Global Theme: Breeze or install `lightly-qt` from AUR for a modern look
- Icons: Papirus (`sudo pacman -S papirus-icon-theme`)
- Cursors: Breeze or Bibata (`yay -S bibata-cursor-theme`)
- Fonts: Set to Inter or Noto Sans for UI; JetBrains Mono for monospace

```
bash  
  
sudo pacman -S ttf-inter-font ttf-jetbrains-mono noto-fonts noto-fonts-emoji
```

System Settings → Workspace Behaviour

- Screen edges: disable ones you won't use — they trigger accidentally
- Virtual Desktops: set up 4 (one per context: Dev, Writing, Browser, Misc)

System Settings → Shortcuts

- Set Super+Return → launch Konsole
- Set Super+E → launch Dolphin
- Set Super+Space → KRunner (already default)

System Settings → Power Management

- Screen off after: 10 min

- Suspend after: 30 min (adjust to taste)

Compositor (System Settings → Display → Compositor)

- Rendering backend: OpenGL 3.1
 - Scale method: Smooth
 - Latency: Balance of speed and smoothness
-

Phase 4 — Productivity Software Stack

4.1 Development tools

bash

```
sudo pacman -S \  
    base-devel \  
    git \  
    github-cli \  
    docker \  
    docker-compose \  
    python \  
    python-pip \  
    nodejs \  
    npm \  
    rustup \  
    go \  
    jq \  
    httpie \  
    neovim \  
    tmux \  
    zsh \  
    fzf \  
    ripgrep \  
    fd \  
    bat \  
    eza \  
    zoxide \  
    htop \  
    btop  
  
sudo systemctl enable --now docker  
sudo usermod -aG docker yourname # replace with your username
```

bash

```
# Rust toolchain
rustup default stable

# Node version manager (more flexible than system node)
yay -S nvm

# Add to ~/.bashrc or ~/.zshrc:
# source /usr/share/nvm/init-nvm.sh
```

Code editor — VS Code:

```
bash

yay -S visual-studio-code-bin
# Or open-source build:
sudo pacman -S code
```

Recommended VS Code extensions for this stack:

- GitLens, Git Graph
- Prettier, ESLint
- Python, Pylance
- Rust Analyzer
- REST Client (replaces Postman for most use cases)
- Todo Tree
- Material Icon Theme

Terminal setup — Zsh + Starship:

```
bash
```

```
sudo pacman -S zsh starship
chsh -s /bin/zsh

# Install Oh My Zsh (optional but useful)
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"

# Install useful plugins
yay -S zsh-autosuggestions zsh-syntax-highlighting

# Add to ~/.zshrc:
# plugins=(git docker zsh-autosuggestions zsh-syntax-highlighting fzf)
# eval "$(starship init zsh)"
# eval "$(zoxide init zsh)"
# alias ls='eza --icons'
# alias cat='bat'
# alias cd='z' # zoxide
```

4.2 Writing and documents

bash

```
sudo pacman -S \
  libreoffice-fresh \
  obsidian \           # notes / knowledge base
  zathura \            # lightweight PDF viewer
  zathura-pdf-mupdf \
  pandoc \             # document conversion (md → pdf, docx, etc.)
  texlive-basic \      # LaTeX for pandoc PDF output
  hunspell \           # spell check
  hunspell-en_us

yay -S typora # if you want a polished Markdown editor
```

Fonts for documents:

bash

```
sudo pacman -S \
  ttf-liberation \
  ttf-dejavu \
  adobe-source-serif-fonts \
  adobe-source-sans-fonts
```

4.3 Browser

bash

```
# Firefox (recommended – best KDE integration, privacy defaults)
sudo pacman -S firefox

# Or Brave:
yay -S brave-bin

# Or Chromium (open source Chrome):
sudo pacman -S chromium
```

Firefox recommended extensions:

- uBlock Origin
- Bitwarden
- Tree Style Tab (vertical tabs — huge productivity gain)
- Tabliss (new tab page)

4.4 Communication

```
bash

yay -S \
    slack-desktop \
    zoom \
    teams-for-linux    # or use the browser version

sudo pacman -S thunderbird    # email client
```

4.5 Utilities

```
bash

sudo pacman -S \
    flameshot \           # better screenshots than Spectacle
    copyq \               # clipboard manager
    keepassxc \           # password manager (local)
    syncthing \           # file sync without cloud
    timeshift \           # system snapshots (works with btrfs)
    kdeconnect \          # phone integration
    filelight \           # disk usage visualiser
    partitionmanager \    # KDE partition tool

yay -S \
    ulauncher \           # alternative to KRunner if you prefer it
    anydesk               # remote desktop if needed
```

Enable syncthing:

```
bash
```

```
systemctl --user enable --now syncthing
```

Phase 5 — System Hardening and Maintenance

5.1 Snapshots with Timeshift

```
bash
```

```
sudo pacman -S timeshift
# Launch Timeshift, select BTRFS mode
# Set schedule: 1 daily, 3 weekly, 2 monthly
# This uses your @snapshots subvolume automatically
```

For automatic pre-update snapshots:

```
bash
```

```
yay -S timeshift-autosnap
# Now every pacman upgrade automatically creates a snapshot first
```

5.2 Paccache — clean old packages

```
bash
```

```
sudo pacman -S pacman-contrib
sudo systemctl enable --now paccache.timer
# Keeps last 3 versions of each package, runs weekly
```

5.3 Firewall

```
bash
```

```
sudo pacman -S ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo systemctl enable ufw
```

5.4 SSD health

```
bash
```

```
sudo pacman -S smartmontools
sudo smartctl -a /dev/nvme0n1    # check health
# fstrim.timer (enabled in phase 2) handles regular TRIM
```

5.5 Failed login limits

```
bash

sudo vim /etc/security/faillock.conf
# Set: deny = 5
# Set: unlock_time = 300
```

Phase 6 — KDE Productivity Tweaks

These are the ones that actually change how you work day to day.

6.1 KDE Activities (underused, very powerful)

Activities are separate workspaces with their own wallpaper, widgets, and recent files. Think of them as work profiles.

Set up 3 activities:

- **Dev** — dark wallpaper, VS Code pinned, terminal shortcuts
- **Writing** — calm wallpaper, Obsidian + LibreOffice pinned
- **Admin** — browser + email + Slack

Switch with Meta+Tab. Each activity remembers open applications.

6.2 Krunner plugins worth enabling

System Settings → KRunner → enable:

- Calculator (just type $=2+2$)
- Unit converter (100 USD in INR)
- Browser history
- SSH connections
- Spell checker

6.3 Clipboard manager

```
bash
```

```
# CopyQ gives you a searchable clipboard history
sudo pacman -S copyq
# Set it to start at login: System Settings → Autostart → Add application
# Bind to Meta+V for quick access
```

6.4 Window tiling without switching WM

KDE 6 has built-in tiling (Window Tiling in System Settings → Window Management). Enable it. It gives you a tiling grid without going full i3.

Alternatively install KWin tiling script:

```
bash

yay -S kwin-bismuth
# Bismuth adds proper tiling behaviour to KDE
```

6.5 Global menu bar (saves vertical space)

System Settings → Startup and Shutdown → Background Services → enable Global Menu.

Then add the "Application Menu Bar" widget to your panel. App menus move to the panel like macOS, freeing window chrome space.

6.6 Useful KDE keyboard shortcuts to set

Shortcut	Action
Meta+Enter	Open terminal
Meta+E	Open Dolphin
Meta+Space	KRunner
Meta+D	Show desktop
Meta+1-4	Switch virtual desktop
Meta+Shift+1-4	Move window to desktop
Meta+Left/Right	Tile window left/right
Meta+Up	Maximise window
Ctrl+F12	Show desktop grid
Alt+F3	Window operations menu

Phase 7 — Dotfiles and Backup Strategy

7.1 Track dotfiles with git

```
bash

mkdir ~/.dotfiles
git init --bare ~/.dotfiles

# Add alias to ~/.zshrc:
alias dotfiles='/usr/bin/git --git-dir=$HOME/.dotfiles/ --work-tree=$HOME'

dotfiles config status.showUntrackedFiles no

# Track files:
dotfiles add ~/.zshrc ~/.config/starship.toml ~/.config/konsolerc
dotfiles commit -m "initial dotfiles"
dotfiles remote add origin git@github.com:you/dotfiles.git
dotfiles push -u origin main
```

7.2 Key config files to back up

```
~/.zshrc
~/.config/starship.toml
~/.config/konsole/
~/.config/kdeglobals
~/.config/kwinrc
~/.config/plasma-org.kde.plasma.desktop-appletsrc
~/.config/kglobalshortcutsrc
~/.local/share/konsole/          # terminal profiles
~/.config/Code/User/settings.json
~/.config/Code/User/keybindings.json
```

7.3 Automated backup with rsync

```
bash
```



```
# Simple off-site backup script – save as ~/bin/backup.sh
#!/bin/bash
set -e
DEST="user@yourserver:/backups/arch"
rsync -avz --delete \
  --exclude='.cache' \
  --exclude='node_modules' \
  --exclude='.local/share/Trash' \
  ~/Documents ~/Projects ~/dotfiles \
  $DEST
echo "Backup complete: $(date)"

chmod +x ~/bin/backup.sh

# Schedule weekly with systemd timer (preferred over cron on Arch)
```

Quick Reference — Commands to Know

```
bash
```

```
# System update (do this weekly)
sudo pacman -Syu

# Update including AUR
yay -Syu

# Search for a package
pacman -Ss keyword
yay -Ss keyword

# Remove package + unused deps
sudo pacman -Rns packagename

# List explicitly installed packages
pacman -Qe

# Find which package owns a file
pacman -Qo /usr/bin/something

# Rollback to a snapshot
# Boot → timeshift → restore → reboot

# Check systemd service logs
journalctl -u servicename -f

# Check what's eating disk
ncdu ~ # (sudo pacman -S ncdu)
```

Recommended Maintenance Schedule

Frequency	Task
Weekly	<code>yay -Syu</code> — full system update
Weekly	Review Timeshift snapshots, delete old ones
Monthly	<code>paccache -r</code> — clean package cache (or let the timer do it)
Monthly	<code>sudo smartctl -a /dev/nvme0n1</code> — SSD health check
Monthly	Test a snapshot restore on a spare partition or VM
Quarterly	Review and rotate SSH keys
Quarterly	Audit installed packages: <code>pacman -Qdt</code> (orphans)

Troubleshooting

Plasma doesn't start after update

```
bash

# Boot to TTY (Ctrl+Alt+F2 at login screen)
# Check SDDM logs:
journalctl -u sddm -b
# Most common fix after nvidia driver update:
sudo mkinitcpio -P && reboot
```

Audio not working

```
bash

systemctl --user status pipewire pipewire-pulse wireplumber
systemctl --user restart pipewire pipewire-pulse wireplumber
# Check audio devices:
wpctl status
```

Screen tearing on Intel/AMD

```
bash

# Add to /etc/X11/xorg.conf.d/20-intel.conf (Intel):
Section "Device"
    Identifier "Intel Graphics"
    Driver "intel"
    Option "TearFree" "true"
EndSection
```

Slow boot — diagnose:

```
bash

systemd-analyze blame | head -20
systemd-analyze critical-chain
```

AUR build fails

```
bash
```

```
# Clean build:  
yay -S --cleanbuilds packagename  
# Check if dependencies are outdated:  
sudo pacman -Syu && yay -S packagename
```

Last step: Once everything is stable, create a Timeshift snapshot labelled "clean-install" — your recovery baseline if anything goes wrong later.